Laboratorio di Calcolo, Esercitazione 4, 5-6 novembre 2026

Canale Q-Z, Docenti: Shahram Rahatlou, Alessandra Betti

Lo scopo di questa esercitazione è calcolare la radice quadrata di un numero positivo con due metodi iterativi, far scrivere al programma in C il risultato di ogni iterazione su un file e disegnare con Python la convergenza dei metodi. Aggiungete un pezzo alla volta, compilando ed eseguendo dopo ogni passo; nella Parte 2 partite da una copia (`cp`) del programma della Parte 1.

► Preparazione

Fate login con `lcsrXXX` (il numero del vostro gruppo), create la cartella `ESER4` ed entrateci. Compilate con `gcc -Wall -o babilon.exe babilon.c -lm` (con `#include <math.h>`). I programmi Python si eseguono con `python3 nome.py`, gli eseguibili con `./babilon.exe`. Nei messaggi stampati non usate lettere accentate né apostrofi. La funzione `fabs(x)` di `math.h` calcola $|x|$ e `sqrt(x)` calcola $\sqrt{x}$.

Il metodo babilonese. La radice quadrata $\sqrt{a}$ di un numero $a > 0$ è il limite della successione

$$r_{n+1} = \frac{1}{2}\left(r_n + \frac{a}{r_n}\right).$$

Il primo termine $r_0 > 0$ si sceglie a piacere: cambiandolo cambia solo il numero di iterazioni. Il ciclo termina quando la differenza $|r_{n+1} - r_n|$ tra due termini successivi è minore della precisione ε richiesta.

► Parte 1: il metodo babilonese e il suo grafico (circa 65 minuti)

Scrivete `babilon.c`. I tre input vanno chiesti **separatamente**, ciascuno con un `printf` seguito da uno `scanf`, nell'ordine ε, r_0, a.

1. Chiedete ε e rileggetelo con un `do/while` finché non vale $10^{-6} \le \varepsilon \le 0.1$ (condizione composta con `||`).
2. Chiedete r_0 e rileggetelo finché non vale $r_0 > 0$.
3. Chiedete a e rileggetelo finché non vale $a > 0$.
4. Con un `do/while` implementate il metodo. Stampate prima la riga con $n = 0$ e r_0, poi a ogni iterazione una riga con n e r_n, con il descrittore `"%3d %.12f\n"`. Il ciclo si ferma quando $|r_{n+1} - r_n| < \varepsilon$.
5. Dopo il ciclo stampate il numero N_{tot} di iterazioni eseguite e la differenza $|r_n - \sqrt{a}|$ tra il risultato e il valore di `sqrt()`, con il formato `%.5g`, che mostra anche le differenze molto piccole.
6. Verificate con questi dati (nell'ordine ε, r_0, a):

`0.1 1 7`	$N_{\text{tot}} = 4$	$r_4 = 2.645767044190$, differenza $1.5733 \cdot 10^{-5}$
`0.001 1 7`	$N_{\text{tot}} = 5$	$r_5 = 2.645751311111$, differenza $4.6779 \cdot 10^{-11}$
`0.0001 30 7`	$N_{\text{tot}} = 7$	$r_7 = 2.645751311846$, differenza $7.8152 \cdot 10^{-10}$

Con `0.001 1 7` le righe stampate cominciano con $r_0 = 1$, $r_1 = 4$, $r_2 = 2.875$. Con i valori `0.5 0.0000001 0.001 0 -2 1 0 -3 7` i primi due ε, i primi due r_0 e i primi due a sono rifiutati e riletti, e il risultato è quello della seconda riga.

Il file e il grafico. Ora il programma scrive i dati su un file e Python li disegna.

7. In `babilon.c` aprite il file `babilon.txt` in scrittura con `fopen`, controllate che l'apertura sia riuscita (`if (fp == NULL)`, un messaggio e `exit(-1)`) e chiudetelo con `fclose` alla fine.

8. Scrivete nel file con `fprintf`, a ogni iterazione (compresa $n = 0$), una riga con n e r_n separati da uno spazio, con lo stesso formato dello schermo. Nel file ci sono solo numeri: nessun testo. Eseguite con `0.001 1 7`: il file ha 6 righe.

9. Con `emacs` create `convergenza.py` e scrivete il programma del listato qui sotto. Il valore di `A` è lo stesso a usato nel programma C; il grafico mostra la stima r_n in funzione di n e una riga rossa a $\sqrt{a}$. Il formato `'o-'` disegna i punti (`o`) uniti da una linea (`-`).

```
import numpy as np
import matplotlib.pyplot as plt

A = 7.                  # lo stesso valore di a usato nel programma C
n, r = np.loadtxt('babilon.txt', unpack=True)   # una colonna, un array

plt.plot(n, r, 'o-', label='stima babilonese')
plt.axhline(y=np.sqrt(A), color='red', linestyle='--', label='sqrt(a)')
plt.xlabel('iterazione n')
plt.ylabel('stima di sqrt(a)')
plt.title('Metodo babilonese, a = 7')
plt.grid()
plt.legend()
plt.savefig('convergenza.png')       # prima di show
plt.show()
```

Listato 1: Programma `convergenza.py`

Con `0.001 1 7` il grafico ha 6 punti, da $(0, 1)$ a $(5, 2.6458)$, e si vede la stima che si avvicina alla riga rossa; il file `convergenza.png` è salvato nella cartella.

► **Parte 2: un secondo metodo, per l'inverso della radice** (circa 50 minuti)
L'inverso $1/\sqrt{a}$ si ottiene come limite della successione

$$y_{n+1} = \frac{3}{2}\, y_n - \frac{a}{2}\, y_n^3,$$

che nel ciclo non ha divisioni. La stima di $\sqrt{a}$ è $1/y_n$. La successione converge solo se il termine iniziale y_0 è positivo e minore di $\sqrt{3/a}$ (0.6547 per $a = 7$): per valori più grandi può divergere. Copiate `babilon.c` in `radice.c`.

1. Chiedete ε e a come nella Parte 1 (due input separati, nell'ordine ε, a, ciascuno riletto finché non è valido); r_0 non si chiede.

2. Prima prova: fissate nel programma $r_0 = 1$ e partite da $y_0 = 1/r_0$. Con un `do/while` calcolate y_{n+1} e stampate a ogni iterazione n e $1/y_n$ (con `"%3d %.12f\n"`), fermandovi quando $|1/y_{n+1} - 1/y_n| < \varepsilon$. Con `0.001 7` le righe stampate sono 1, −0.5, 0.04, −0.000018298262 e 0 (4 iterazioni), e la differenza da `sqrt(a)` vale 2.6458: $y_0 = 1$ è troppo grande e il risultato è sbagliato.

3. Correggete il punto di partenza: calcolate r_2 con due iterazioni del metodo babilonese a partire da $r_0 = 1$ (due righe nel programma, senza stampare nulla) e partite da $y_0 = 1/r_2$.

4. Dopo il ciclo stampate N_{tot}, il numero di iterazioni del secondo metodo, e la differenza $|1/y_n - \sqrt{a}|$ con `%.5g`.

5. Scrivete su `radice.txt`, come nella Parte 1, una riga con n e $1/y_n$ per ogni iterazione, con $n = 0$ il valore di partenza $1/y_0 = r_2$.

6. Verificate con questi dati (nell'ordine ε, a):

`0.1 7`	$N_{\text{tot}} = 2$	$1/y_2 = 2.646092354490$, differenza $3.4104 \cdot 10^{-4}$
`0.001 7`	$N_{\text{tot}} = 3$	$1/y_3 = 2.645751376987$, differenza $6.5922 \cdot 10^{-8}$
`0.000001 7`	$N_{\text{tot}} = 4$	$1/y_4 = 2.645751311065$, differenza $2.2204 \cdot 10^{-15}$

   Le righe stampate con `0.001 7` sono 2.875, 2.670544337138, 2.646092354490, 2.645751376987.

7. Copiate `convergenza.py` in `confronto.py` e fate leggere sia `babilon.txt` (eseguito con `0.001 1 7`) sia `radice.txt` (eseguito con `0.001 7`). Disegnate sullo stesso grafico i punti del metodo babilonese e i punti $1/y_n$ del secondo metodo con un altro marcatore e un altro colore, più la riga rossa a $\sqrt{a}$. Il secondo metodo parte dopo due passi del primo: disegnatelo in funzione di $n + 2$. Scegliete `xlim` e `ylim` guardando i dati: i primi punti del metodo babilonese (1 e 4) possono restare fuori dalla finestra (per esempio con $1.5 \le n \le 5.5$ e $2.6 \le y \le 3$). Salvate il grafico in `confronto.png`.

► **Esercizio facoltativo, per casa: l'errore in scala logaritmica**
Con il metodo babilonese l'errore $|r_n - \sqrt{a}|$ diminuisce molto in fretta. Per vederlo si usa un grafico con l'asse y logaritmico (`plt.yscale('log')`).

1. Eseguite `babilon.exe` con `0.001 1 7` e copiate il file con il comando `cp babilon.txt babilon_1.txt`; poi eseguitelo con `0.001 30 7` e copiate il file con il comando `cp babilon.txt babilon_30.txt`.
2. Scrivete `errore.py`: leggete i due file, calcolate per ciascuno l'array dell'errore $|r_n - \sqrt{a}|$ con `np.abs` (il valore assoluto di ogni elemento) e `np.sqrt` e disegnate i due errori in funzione di n con `yscale('log')`, con legenda per $r_0 = 1$ e $r_0 = 30$. Salvate il grafico in `errore.png`.
3. Con $r_0 = 1$ l'errore passa da circa $9 \cdot 10^{-3}$ ($n = 3$) a $2 \cdot 10^{-5}$ ($n = 4$) e a $5 \cdot 10^{-11}$ ($n = 5$): a ogni passo il numero di decimali corretti circa raddoppia. Con $r_0 = 30$ servono 7 iterazioni e l'ultimo errore è circa $8 \cdot 10^{-10}$.